KAPLAN

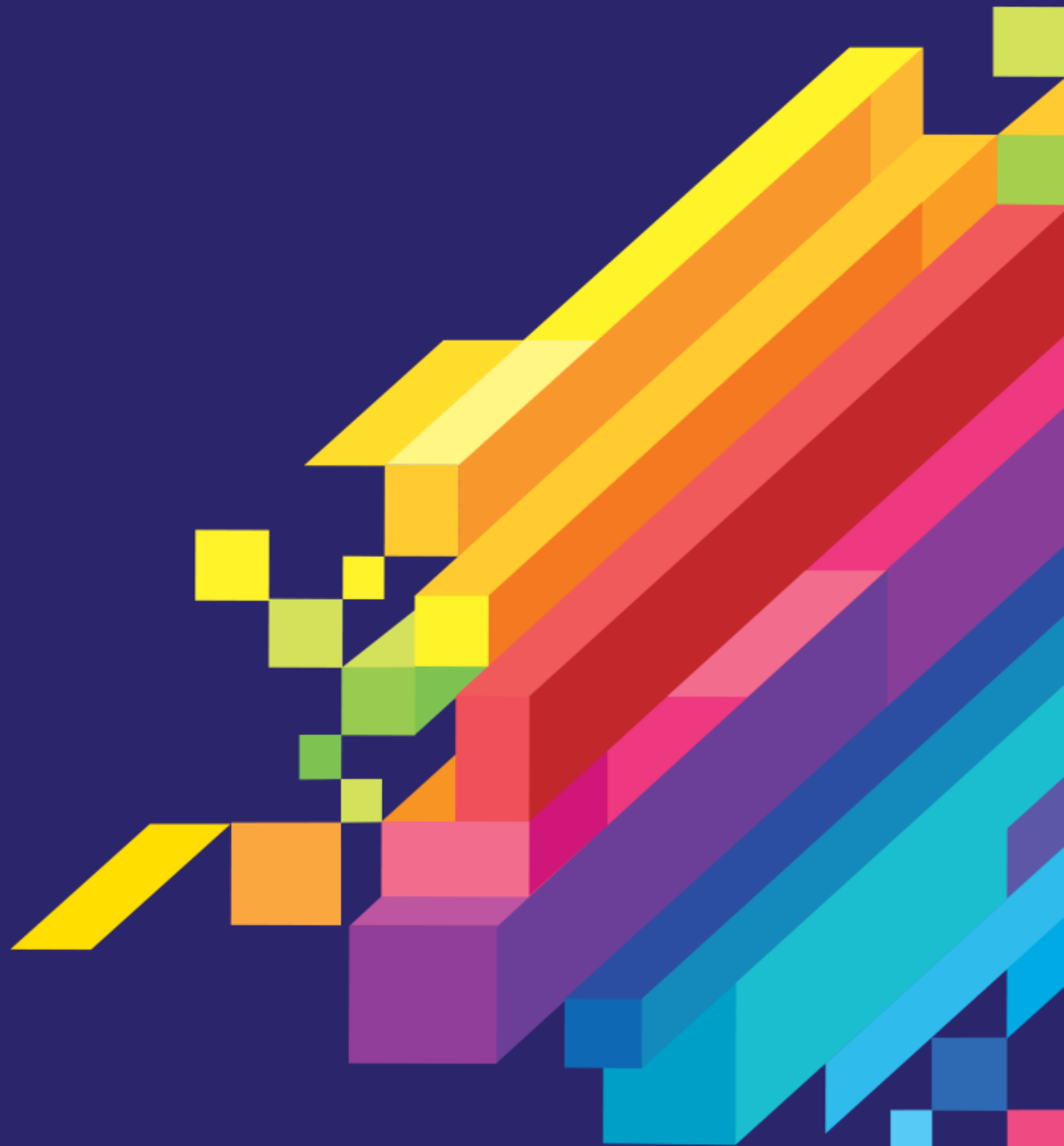
INTRODUCTION TO DATA COMMUNICATIONS

Topic 3: Understanding Transport layer protocols

Our vision is to be Singapore's private education institution of choice, and we are committed to helping our students achieve their career and educational goals through lifelong learning opportunities.

We build futures, one success story at a time.

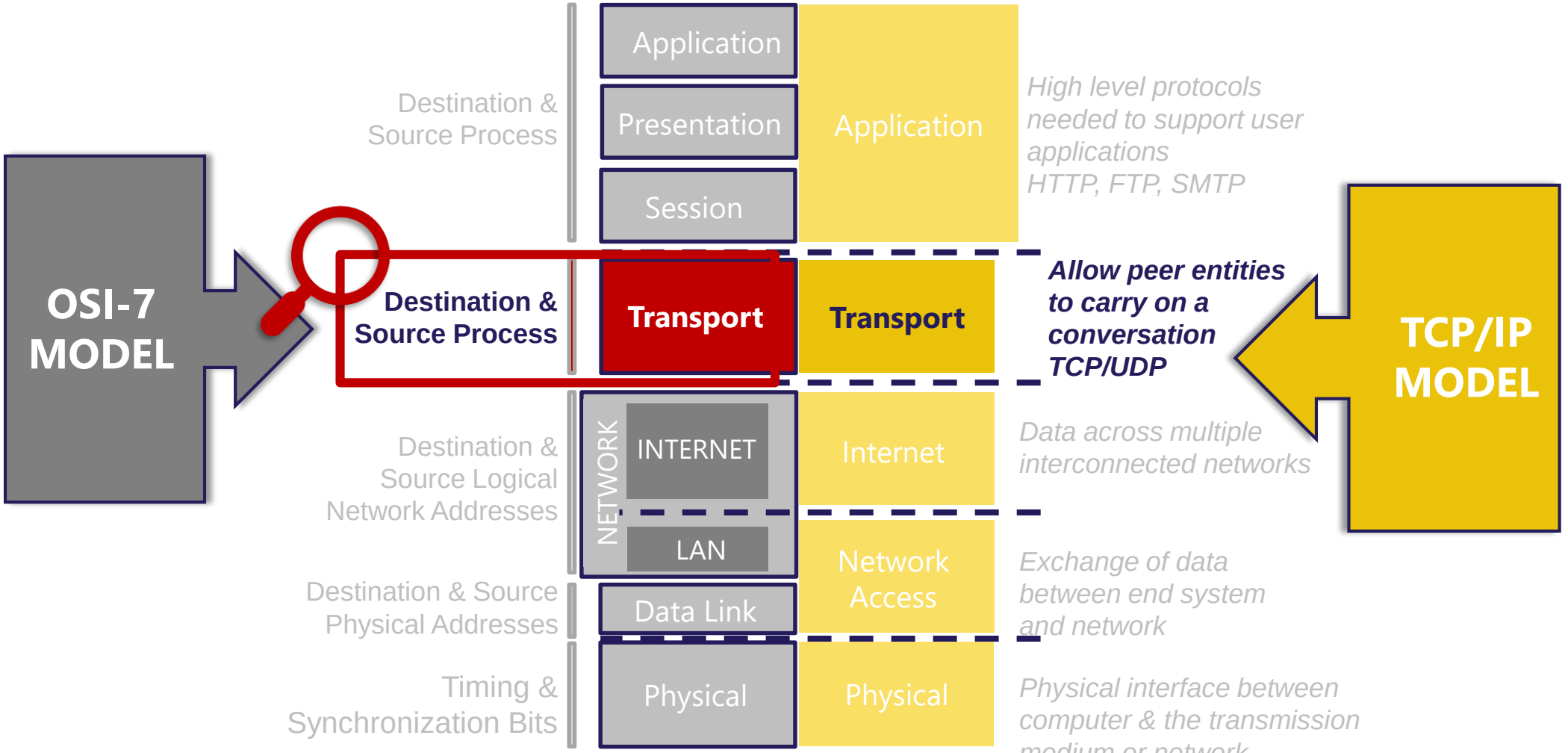
Integrity . Knowledge . Support . Opportunity . Results



Topic 3 – Transport Layer Learning Outcomes

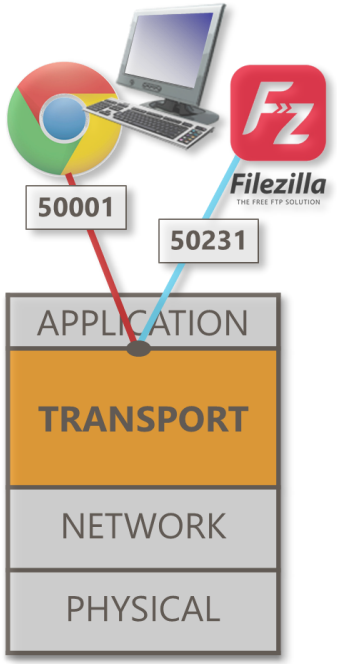
- Addressing
- Connection Establishment
- Connection Release
- Flow Control and Buffering
- Multiplexing
- Crash Recovery

KAPLAN


OSI-7 MODEL
Application
Presentation
Session
Transport
Network
Data Link
Physical
Destination & Source Process
Destination & Source Process
Destination & Source Logical Network Addresses
Destination & Source Physical Addresses
Timing & Synchronization Bits
NETWORK
INTERNET
LAN
Application
Transport
Internet
Network Access
Physical
High level protocols needed to support user applications HTTP, FTP, SMTP
Allow peer entities to carry on a conversation TCP/UDP
Data across multiple interconnected networks
Exchange of data between end system and network
Physical interface between computer & the transmission medium or network
TCP/IP MODEL

Transport Layer



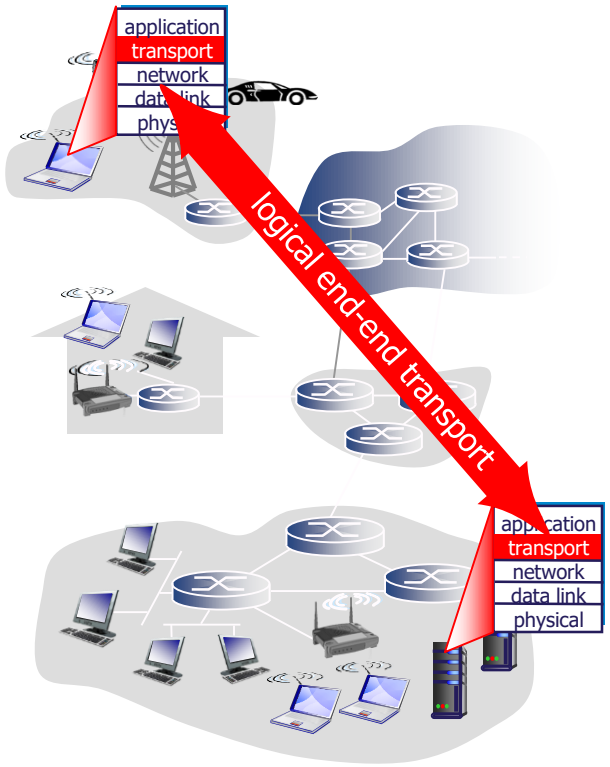


Transport layer provides communication services for a **process** in a host **to** a **process** in another host

KAPLAN

Transport Layer

- **Heart** of whole protocol hierarchy
- Generally **Reliable** data transmission
- **Cost-effective** data transport
- **Independent** of physical network



Why Transport layer?

- Transport layer entity on user's machines, while network layers mostly runs on routers
- To be more reliable than the underlying network service
- **Transport layer primitives** can be implemented as calls to library procedures in order to make them independent of network service primitives

KAPLAN

Transport Layer Elements

- Addressing
- Connection Establishment
- Connection Release
- Flow Control and Buffering
- Multiplexing
- Crash Recovery

KAPLAN

Transport Layer Elements

- **Transport Protocols ~= Data Link Protocols**
  - ✓ *Error control*
  - ✓ *Sequencing*
  - ✓ *Flow control and other issues.*

- *differences in:*
  1. **Addressing**
  2. **Connection Establishment**
  3. **Connection Release**
  4. **Flow Control & Buffering**
  5. **Multiplexing**
  6. **Crash Recovery**

KAPLAN

1. Addressing

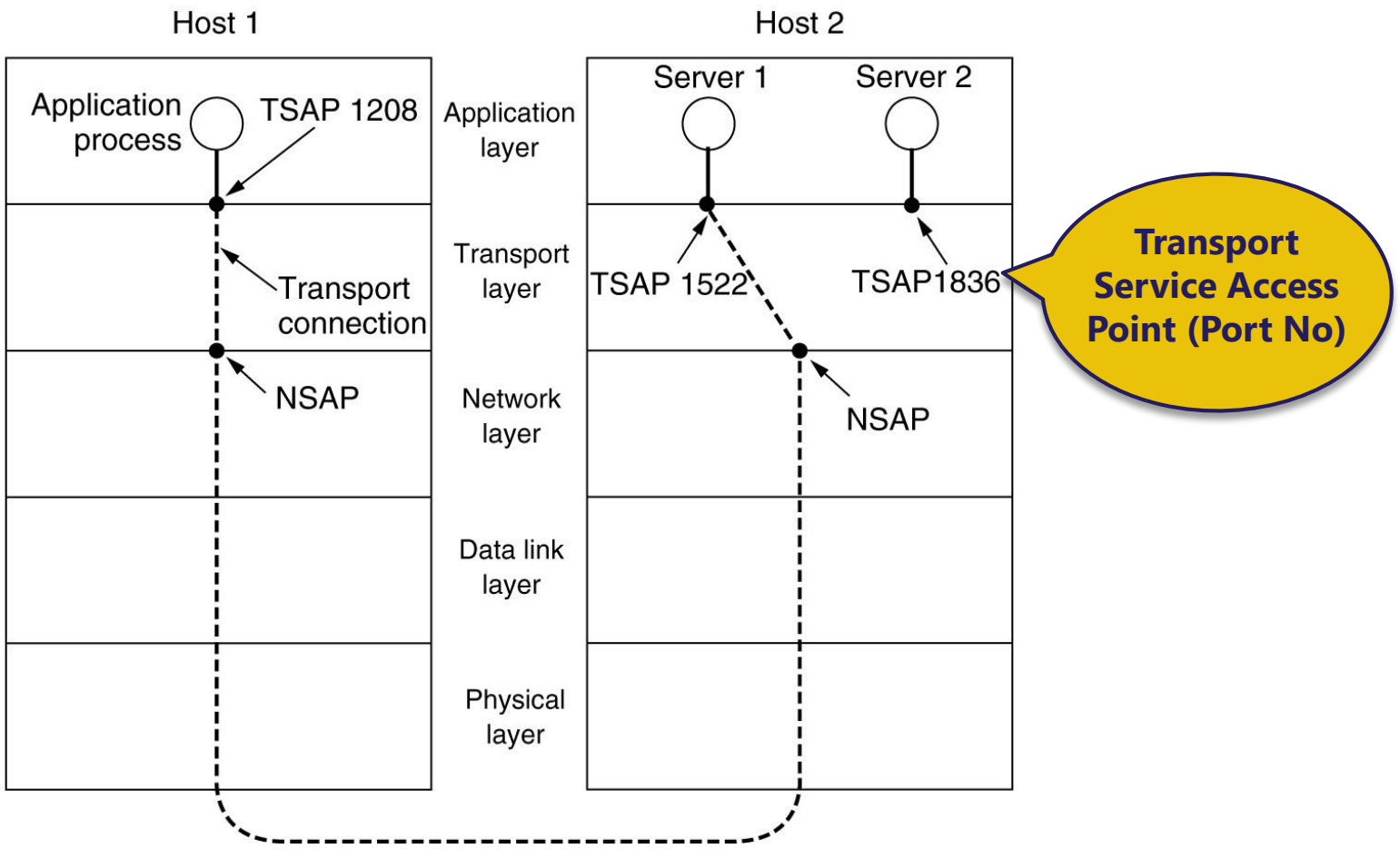
Which application (server) on a remote host **to connect**

✓ *Assign a* **port number** *to Application !*

How to determine the **port number** of a particular application

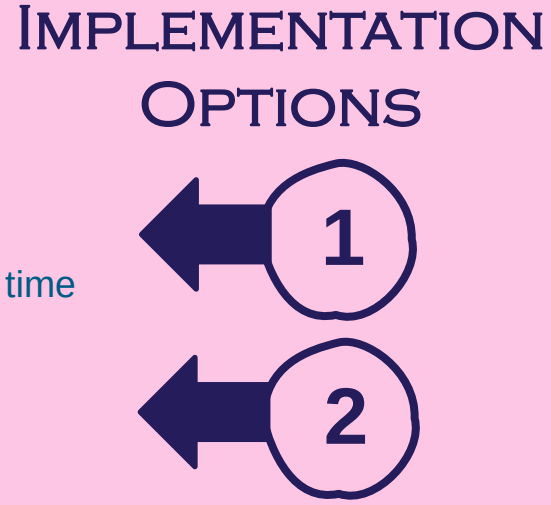


KAPLAN


Host 1
Host 2
Application process
TSAP 1208
Transport connection
NSAP
Application layer
Transport layer
Network layer
Data link layer
Physical layer
Server 1
Server 2
TSAP 1522
TSAP1836
NSAP
Transport Service Access Point (Port No)

1. Addressing: TSAP

- Popular applications use the same TSAP address (permanently) on every host
  - **well-known ports**



- Others use:
  - **Initial Connection Protocol**
    - a special **Process Server** acts as a **proxy** which listen to a set of less heavily used servers at the same time
  - **Directory Server**
    - a.k.a. Name Server
    - Remote user connect to a well known TSAP/port address to resolves symbolic service names into TSAPs
    - Client break connection with the directory server and make a new connection with the required service on the TSAP/port number supplied

Complete TCP address (Socket)

- Port Numbers Ranges [RFC768]:

<IP>: <Port> *or* <NSAP>:<TSAP>

i.e. 192.168.10.5:80

Well Known Ports: 0 - 1023

Assigned by the IANA (Internet Assigned Numbers Authority)

Registered Ports: 1024 - 49151

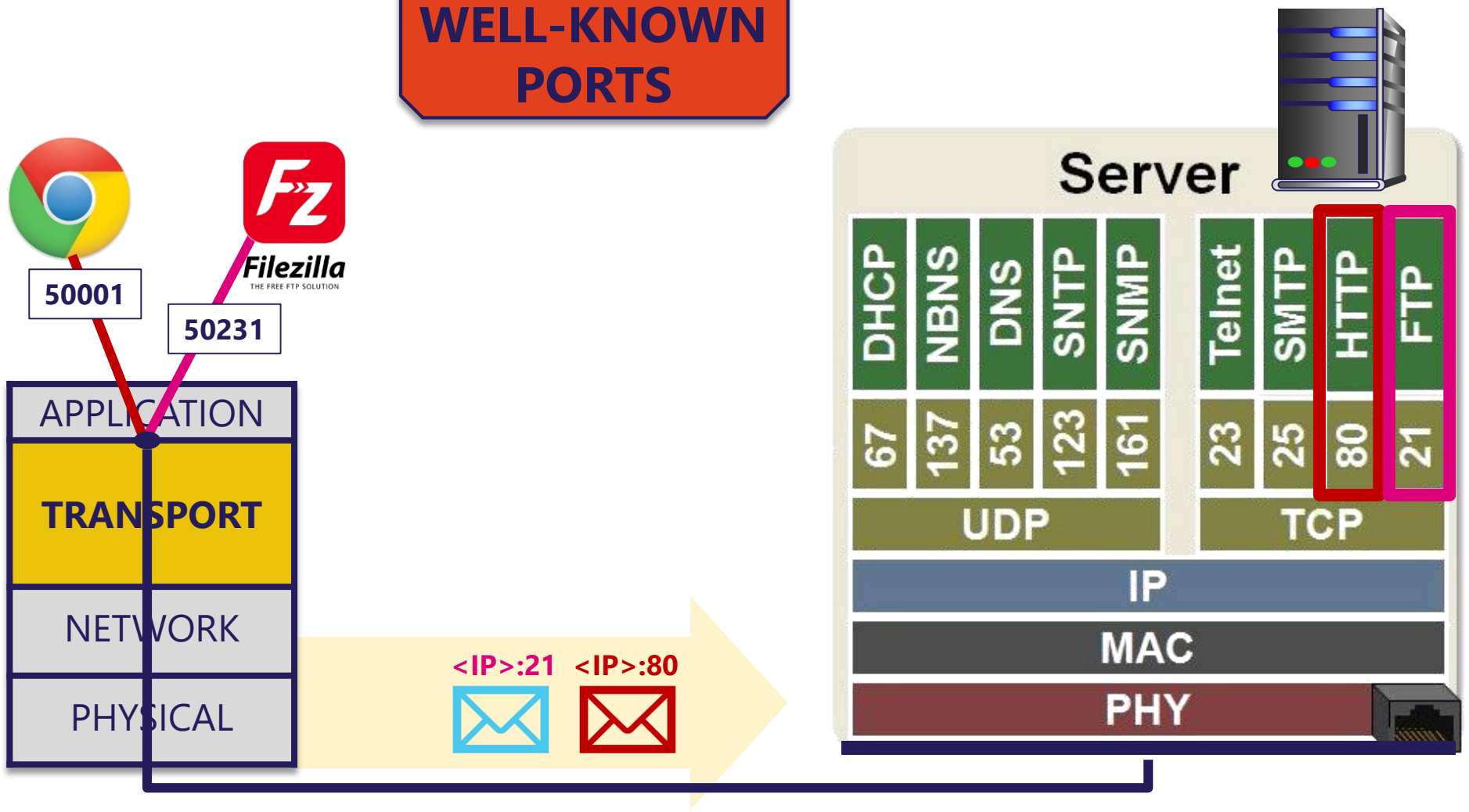
Dynamic / Private Ports: 49152 - 65535

IANA registered, for the convenience of Internet Community

*For the complete list: www.iana.org/assignments/port-numbers

KAPLAN

WELL-KNOWN PORTS



KAPLAN

2. Connection Establishment

- Send a Connection Request **(CR)**
- Wait for a Connection Accepted **(CA)**



network can lose, store, and duplicate packets.

KAPLAN

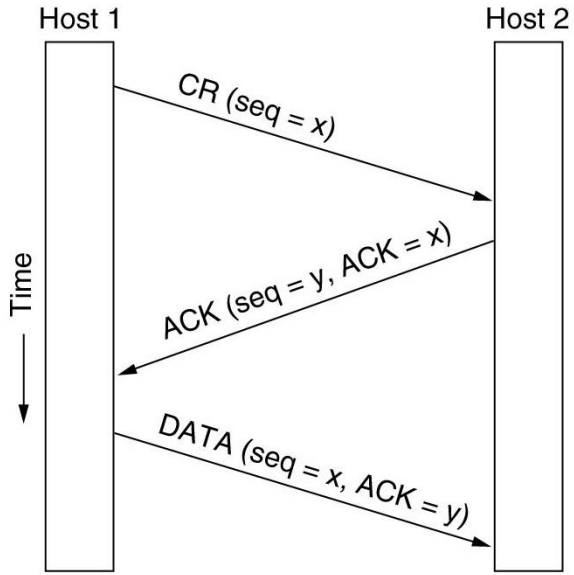
Duplicates

- Ensure packets will not live in the network forever

- **Solutions:**
  - ✓Restrict Packet **lifetime**
  - ✓Restrict **subnet design**
  - ✓Put a **hop counter** in each packet
  - ✓**Time-stamp** each packet

KAPLAN

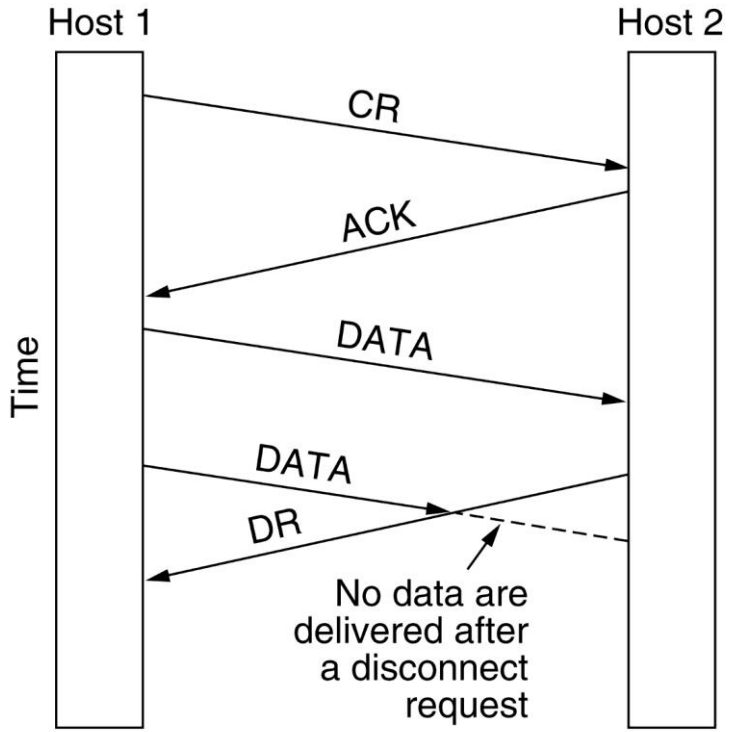
Tomlinson (1975): 3–Way Handshake



3. Connection Release

1. Asymmetric Release

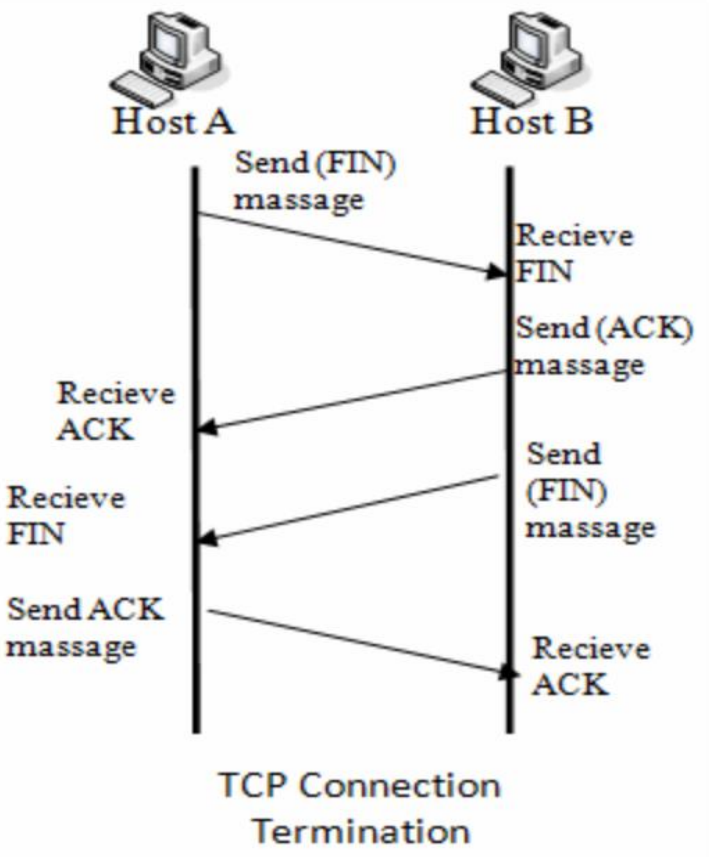
- ✓ Either user can issue a disconnect
- ✓ Rather abrupt, can result in **loss of data**



3. Connection Release – cont.

2. Symmetric Release

Four-way handshake



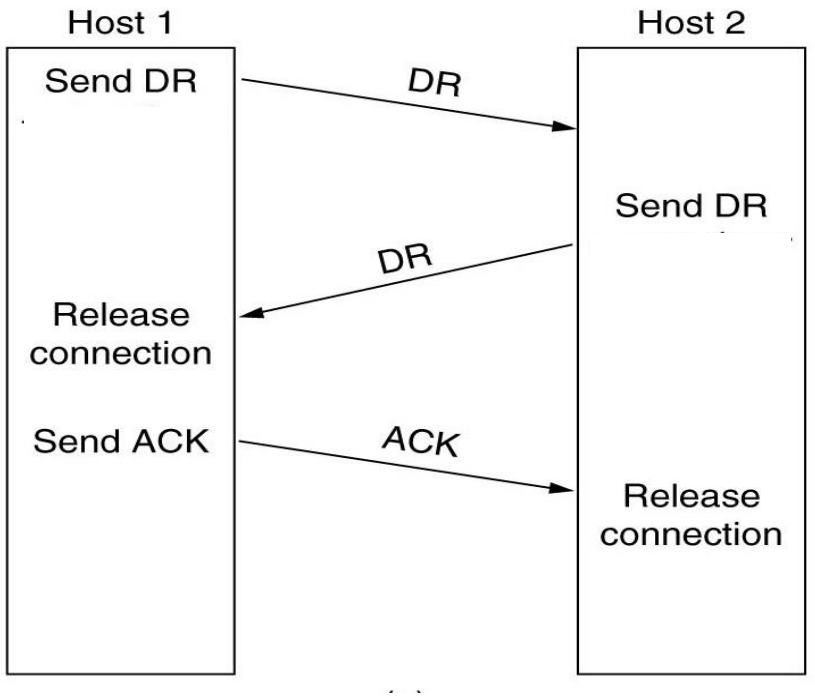
TCP Connection Termination

KAPLAN

3. Connection Release – cont.

2. Symmetric Release

- ✓ each direction of the connection is closed separatel
- ✓ **Not foolproof !** (two-army problem)
- ✓ Can result in one **half** of a connection remaining **op**



Three-way handshake

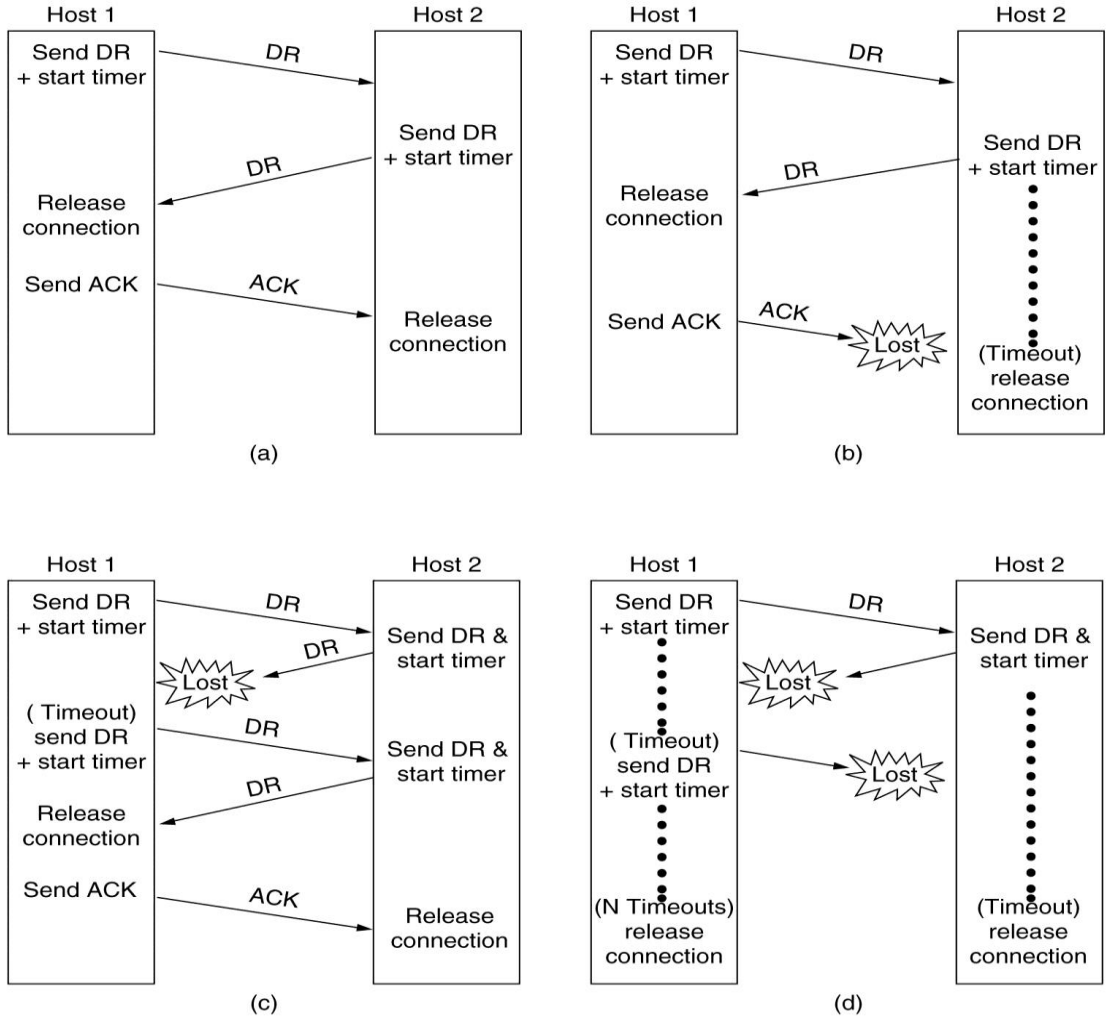
CONN. RELEASE

a) Normal three-way handshake

b) Final ACK lost

c) Response lost

d) Response lost and **subsequent DRs** lost



KAPLAN

4. Flow Control

- Flow control in **TL** ~= Flow control in **DL**
  - ✓ But not the same
  - ✓ Both use sliding window or other scheme

Dynamic Buffer Management

- ✓ decouple the buffering from the acknowledgement
- ✓ a **variable-sized window**



As memory price fall – equip hosts with more memory.
BUT, the **carrying capacity of the subnet is the bottleneck**

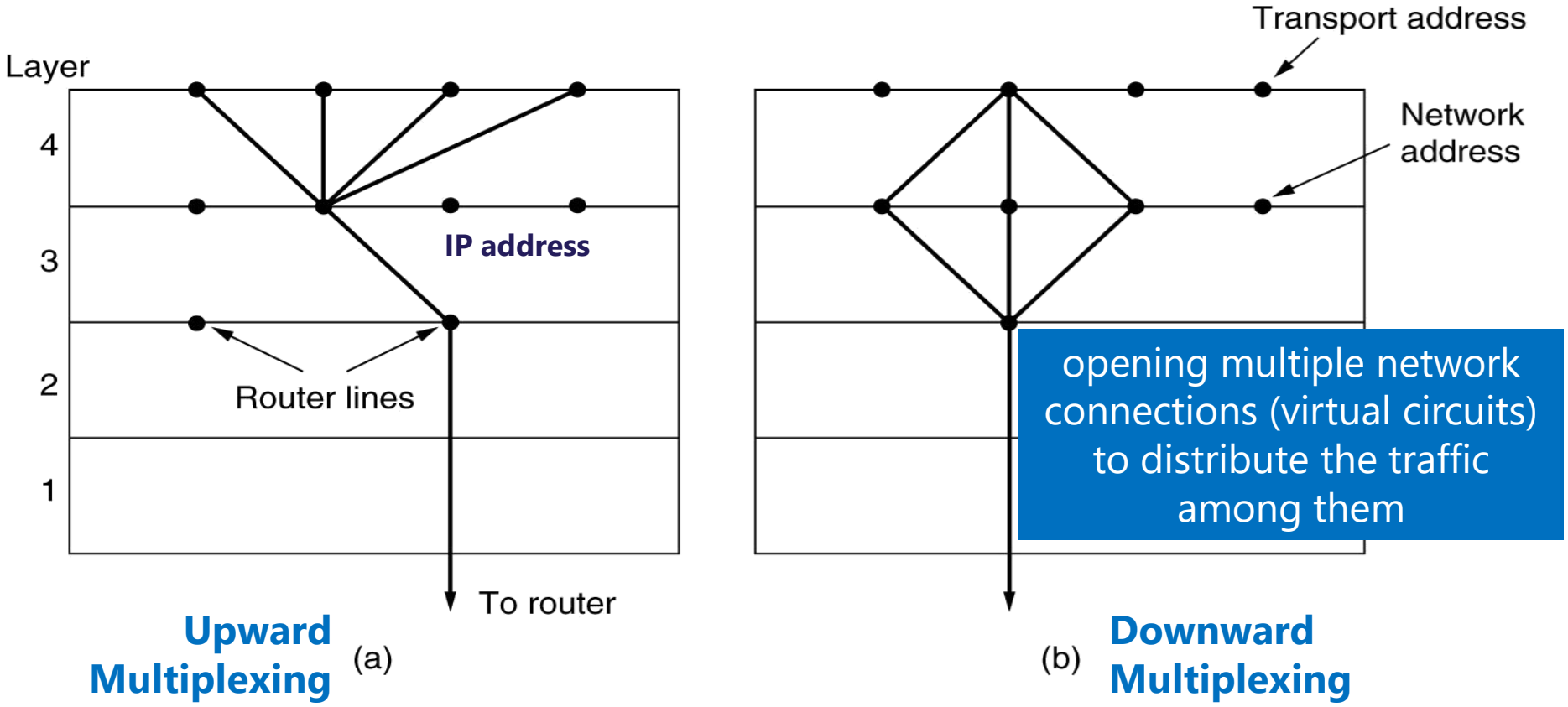


solution: mechanism based on subnet's carrying capacity rather than on the receiver's buffering capacity.

21

KAPLAN

5. Multiplexing



Upward Multiplexing

Downward Multiplexing

opening multiple network connections (virtual circuits) to distribute the traffic among them

KAPLAN

6. Crash Recovery

- **Router / Network Crash**

-For datagram service network:

✓ Transport entities expect lost TPDUs all the time and know how to cope



-For connection-oriented network:

✓ Loss of virtual circuit is handled by establishing a new one and probably the remote transport entity for TPDUs received and not received

KAPLAN

Transport Control Protocol (TCP)

- Fundamentals
- TCP Header
  - Flags (SYN, FIN, ACK, RST)
  - Flag (URG, PSH) – in depth
  - TCP Options
  - Window Size (Dynamic Buffer Management)
- TCP Flow Control
  - Dynamic Buffer Management

KAPLAN

TCP

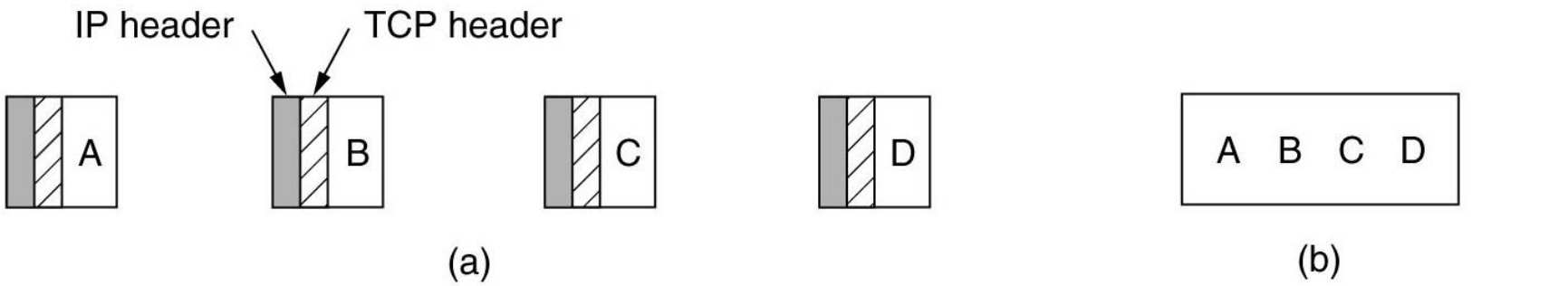
- The (other) **main** transport **protocol** used in the **Internet**
  - ✓ **Connection-oriented** protocol
  - ✓ RFC 793 (formal), RFC 1122 & 1323 (bug fixes)
  - ✓ Provide a reliable end-to-end communication **over** an **unreliable internetwork**
- **Connections** are:
  - ✓ **Full duplex** and point-to-point
  - ✓ **A byte stream** not a message stream



No support for Multicasting or Broadcasting !

KAPLAN

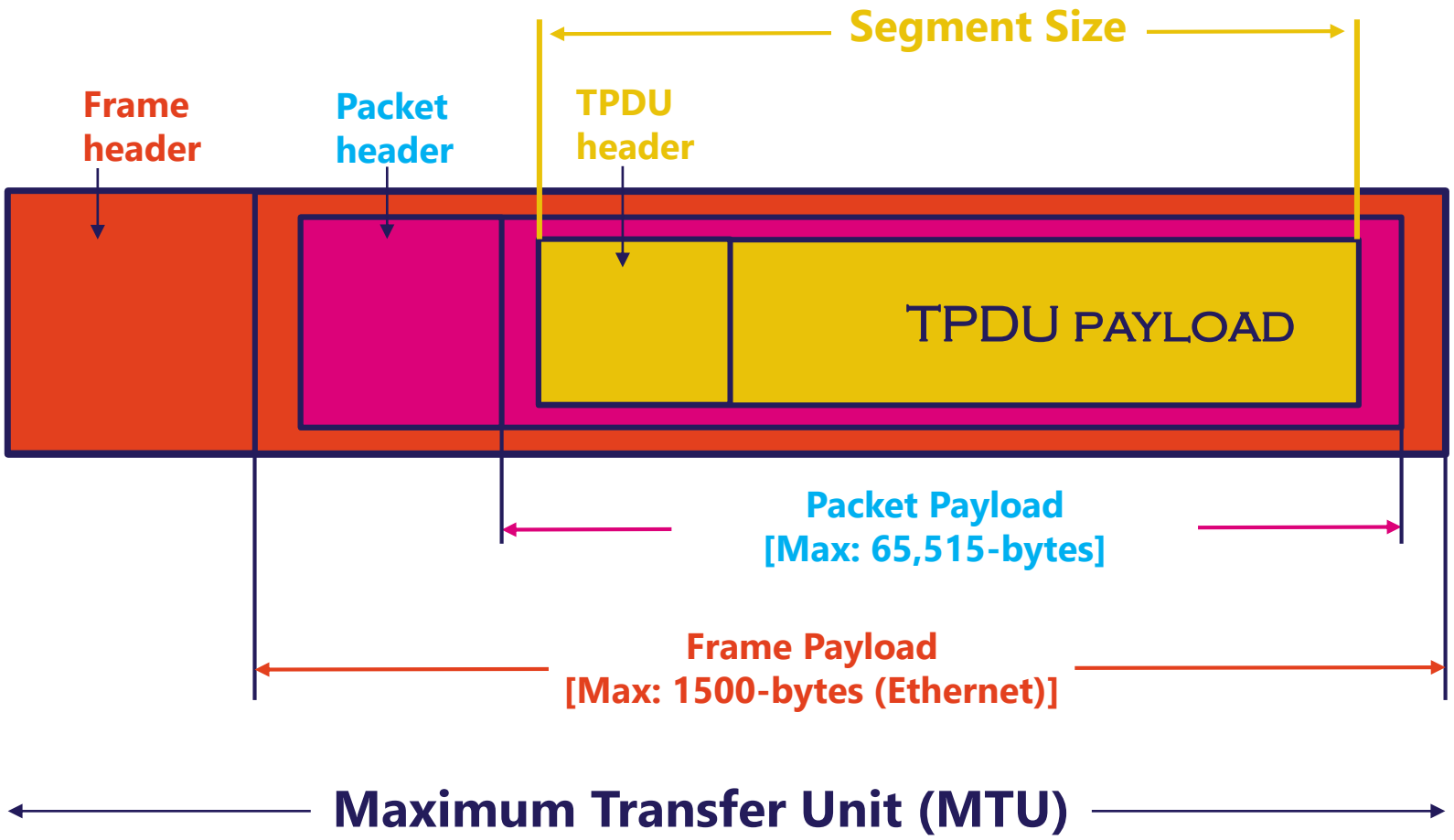
TCP: Header



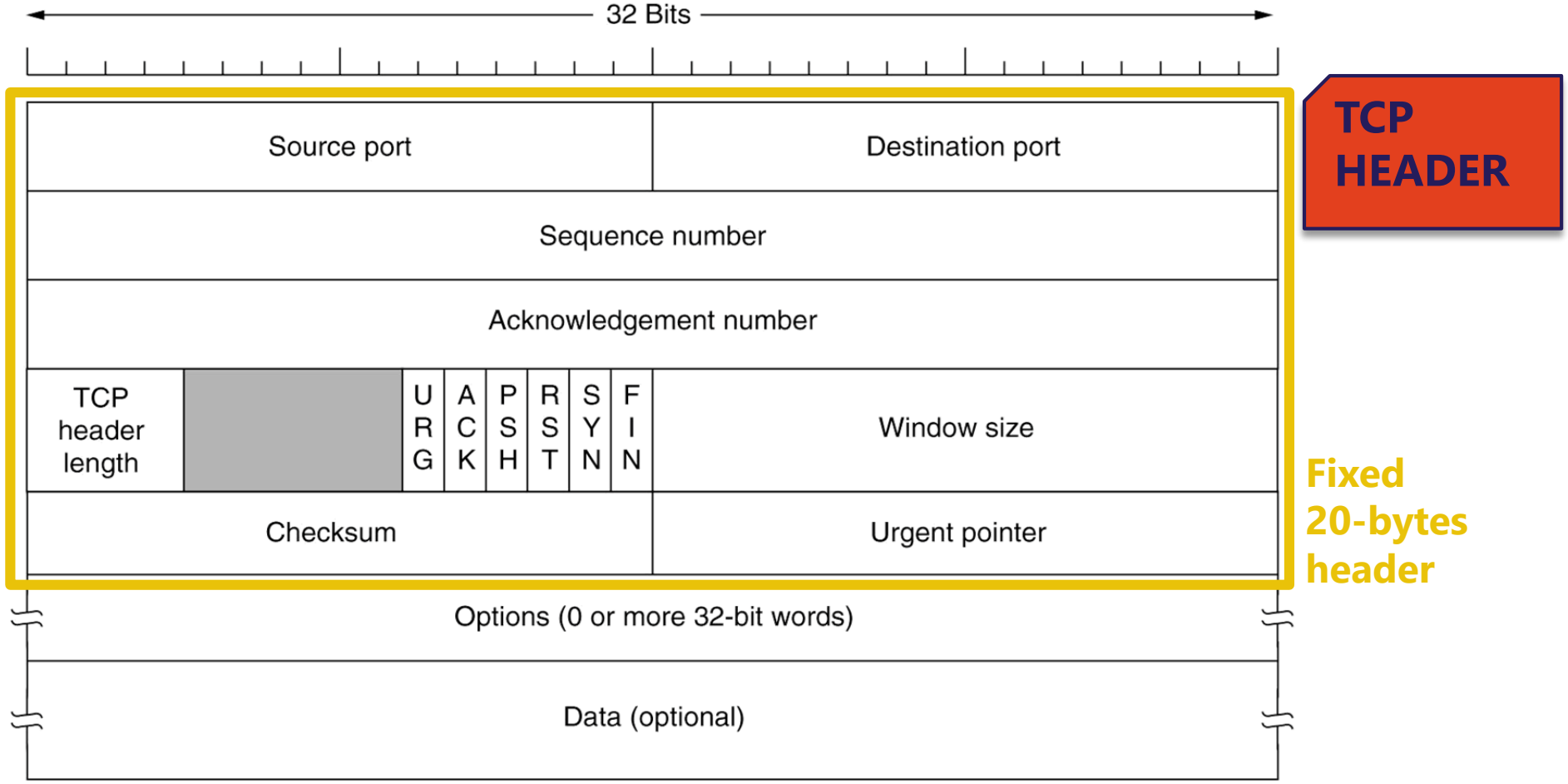
512-byte segments
sent as a separated IP datagrams

2048-bytes of data
received to the application in a single **READ** call

KAPLAN

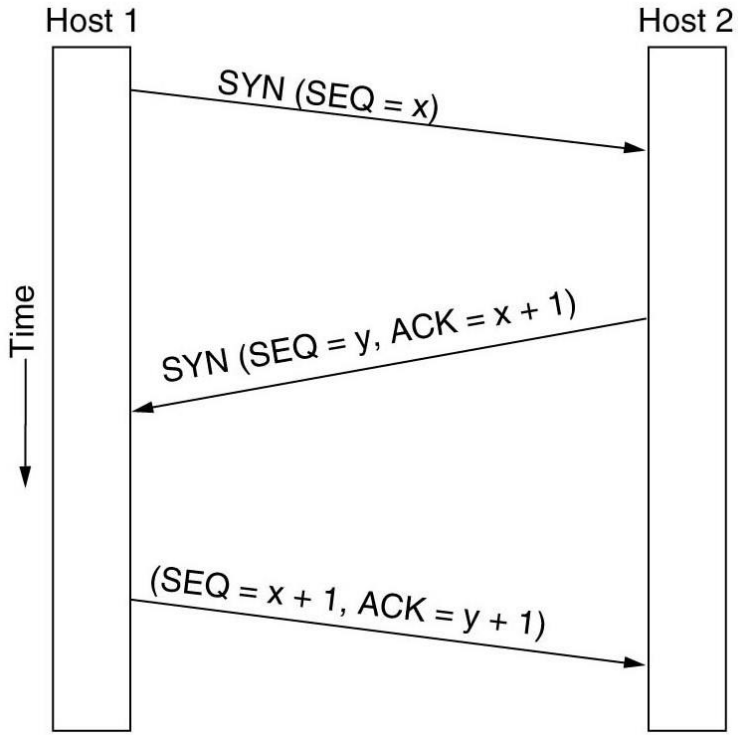

Segment Size
Frame header
Packet header
TPDU header
TPDU PAYLOAD
Packet Payload
[Max: 65,515-bytes]
Frame Payload
[Max: 1500-bytes (Ethernet)]
Maximum Transfer Unit (MTU)

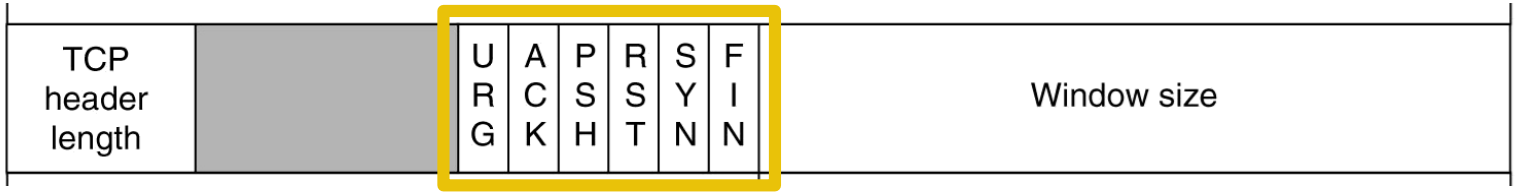
KAPLAN


32 Bits
Source port
Destination port
Sequence number
Acknowledgement number
TCP header length
URG
ACK
PSH
RST
SYN
FIN
Window size
Checksum
Urgent pointer
Options (0 or more 32-bit words)
Data (optional)
TCP HEADER
Fixed 20-bytes header

TCP: Header – cont.

- **Sequence Number (32-bit)**
  - sequence number of the first data octet (byte) in this segment
  - if SYN is set this field is the initial sequence number (ISN) and t
- **Acknowledgement Number (32-bit)**
  - sequence number of the next data octet the TCP entity expects
  - NOTE that TCP is byte or stream oriented.





- ✓ **URG:** Urgent pointer field significant. Inform the destination TCP user that 'urgent' data is arriving.
- ✓ **ACK:** Acknowledgement field significant.
- ✓ **PSH:** Push function. A TCP user can require TCP to send (receive) all outstanding data up to and including that labelled with a **PUSH** flag.
- ✓ **RST:** Reset the connection.
- ✓ **SYN:** Synchronize the sequence numbers. Used to establish connections.
- ✓ **FIN:** No more data from sender. Used to release connections.



FLAGS

KAPLAN

TCP
Implementation Policies

- Send Policy
  - Silly Window Syndrome
- Deliver Policy
- Accept Policy
- Retransmit Policy
- Acknowledge Policy

KAPLAN

TCP: Implementation Policies

- The standard provides a precise specification of the protocol to be used between TCP entities

- Policies defined for the protocol
  1. **Send policy**
  2. **Deliver policy**
  3. **Accept policy**
  4. **Retransmit policy**
  5. **Acknowledge policy**

KAPLAN

1. Send Policy

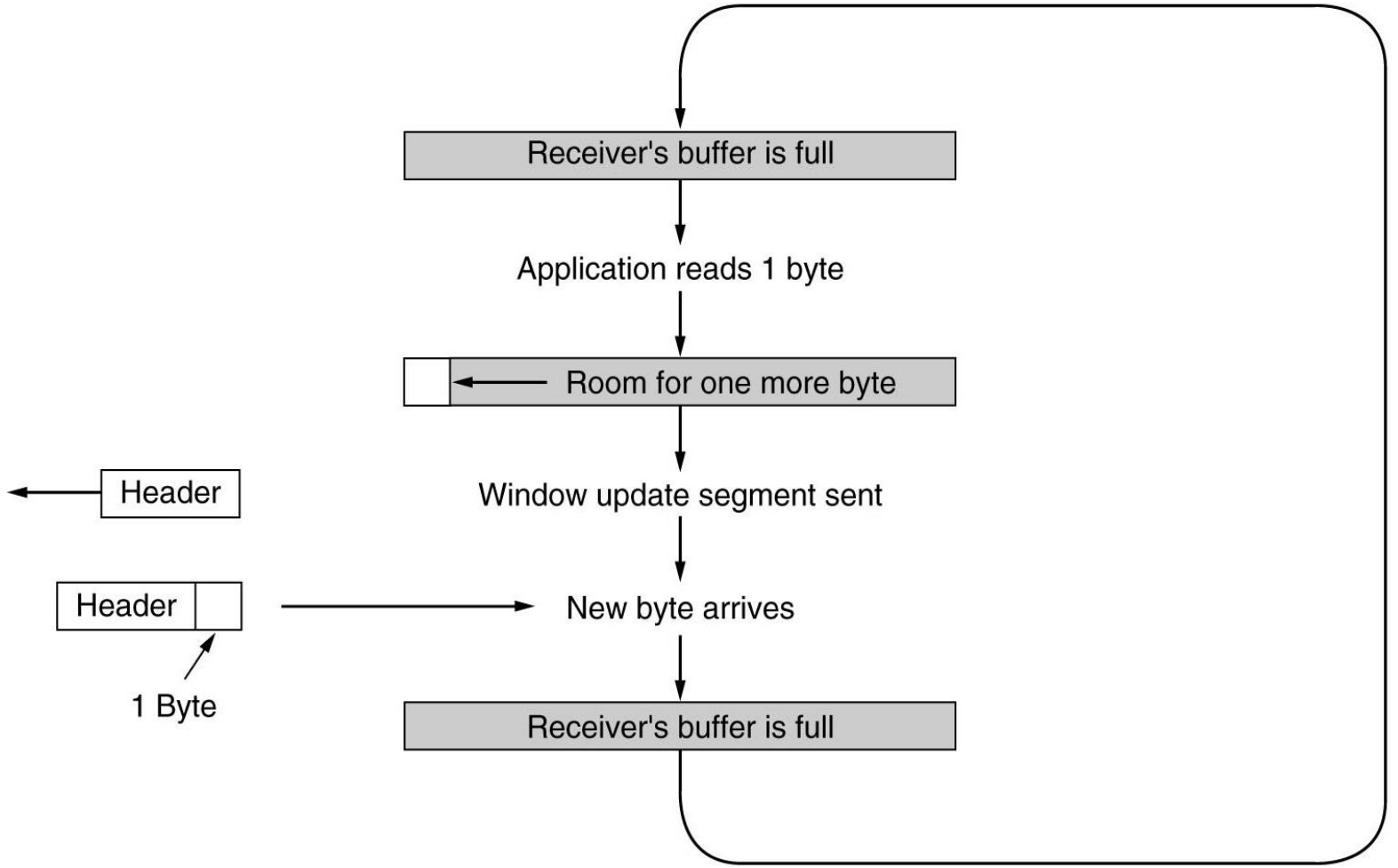
In the **absence of PUSH** data and a closed transmission window, a sending TCP entity is free to **transmit data at its own convenience**

- depend on performance considerations

  ✓ if **infrequent and large** *(buffer @ sender)*
  - low overheads
  - slow response

  ✓ if **frequent and small** *(buffer @ receiver)*
  - quick response
  - high overheads
  - silly window syndrome

KAPLAN



Silly Window Syndrome

KAPLAN

2. Deliver Policy

Too frequent delivery means **too many OS interrupts**

- Arriving data are stored in deliver buffer

✓ **if PUSH flag is set**

Data along with any other data in the deliver buffer are submitted to the destination application in a *RECEIVE* command.

✓ **if PUSH flag is not set**

TCP may wait, *e.g. to avoid excess interrupts*

✓ **if URG flag is set**

The receiving application is signaled that urgent data is present

KAPLAN

3. Accept Policy

- **In Order**
  - ✓ **Discard** out of order segments

- **In Window**
  - ✓ **Accept** all segments within the receive window
  - ✓ Complex acceptance test and sophisticated storage



KAPLAN

4. Retransmit Policy

TCP maintains a **queue of segments** that have been sent but **not ACKed**

- **First Only** suitable for **in-window accept policy**

✓ one retransmission timer for the entire queue

✓ if an ACK is received, the segment/s removed and timer reset

✓ if timer expires, first segment in the queue is retransmitted

Selective-Repeat ARQ

- **Batch** suitable for **in-order accept policy**

✓ same as above, except when timer expires, retransmit all segments in the queue

Go-Back-N ARQ

- **Individual** suitable for **in-window accept policy**

✓ one timer for each segment

Selective-Repeat ARQ

KAPLAN

5. Acknowledge Policy

- **Immediate**
- **Cumulative**

✓wait for an outbound segment, piggyback the ACK

✓Timer to avoid long delay

KAPLAN

Thank you

